File I/O in Java

Seven exercises, the bonus, and verified program output

Completed solutions

1. Generate and reread 500 random numbers.
2. Create n CSV files and find minimum / maximum.
3. Save three team members as CSV.
4. Read students into an ArrayList and display them.
5. Add three students, sort A-Z, and save.
6. Analyze candidate scores and gender statistics.
7. Export groups with summary footer rows.

Bonus. Serialize and deserialize Student objects.

Execution: Java 17 runtime; Eclipse ECJ 3.40.0 compiler targeting Java 17. The source requires JDK 17 or newer and no external Java libraries.

Evidence: Browser screenshots render the repository source and genuine captured standard output. Interactive inputs are listed separately. Random values differ on each run. All student names and IDs in the demonstrations are fictional.

Supplied dataset - verified aggregates

Population	Total	Male	Female
All candidates	958	467 (48.75%)	491 (51.25%)
Average >= 90	48	14 (29.17%)	34 (70.83%)
Group A	81	50 (61.73%)	31 (38.27%)
Group B	185	84 (45.41%)	101 (54.59%)
Group C	308	134 (43.51%)	174 (56.49%)
Group D	249	130 (52.21%)	119 (47.79%)
Group E	135	69 (51.11%)	66 (48.89%)

Assignment details resolved: Exercises 4-5 accept the exercise 3 filename as a fallback. Exercise 7 also exports group E, preserving all 958 candidates. Gender percentages use each displayed population as the denominator.

Local dataset: Individual candidate records and group exports remain local. Only aggregate results appear in this report.

Repository: github.com/anupheap/PDI-Lab4

Exercise 1 - Lucky numbers

Source screenshot. Generate, save, and reread 500 integers.

PDI / LAB 04 JAVA SOURCE

src/Exercise1.java | lines 1-31

```java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Random;

/** Writes 500 random integers to CSV, then displays the values read from disk. */
public class Exercise1 {
    public static void main(String[] args) throws IOException {
        Path file = Path.of("LuckyNumber.csv");
        Random random = new Random();
        try (BufferedWriter writer = Files.newBufferedWriter(file, StandardCharsets.UTF_8)) {
            for (int i = 0; i < 500; i++) {
                writer.write(Integer.toString(random.nextInt(100) + 1));
                writer.newLine();
            }
        }

        System.out.println("Saved 500 numbers to " + file + ". Reading them back:");
        int count = 0;
        try (BufferedReader reader = Files.newBufferedReader(file, StandardCharsets.UTF_8)) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.printf("%3d%s", Integer.parseInt(line), ++count % 20 == 0 ? "\n" : " ");
            }
        }
        System.out.println("Numbers read: " + count);
    }
}
```

Exercise 1 - Lucky numbers

Program output screenshot. Generate, save, and reread 500 integers.

PDI / LAB 04

CAPTURED CONSOLE OUTPUT

Exercise1 | successful run

```
$ java -cp build Exercise1

CAPTURED STDOUT
Saved 500 numbers to LuckyNumber.csv. Reading them back:
 14  47  63  76  45  69  66  20  80  60  99  71  70  93  44  37   3  53  28  38
 47  52  31  33   3  30   7  67  44   9   9  68  17   3  79  82  45  14  65  99
 85  31  34   2  16  39  48  96   8  96  80  92  71  95  88  58  52   3  96  64
  7   8  92  83   7   3   5   9  85  30  43  91  47  50  30  27  69   4  40  28
  4  85  34  30  77  47  24  38  80  38  68  42  29   8  67  29  45  82  51  18
 46  55  73  11  10  26  46  91  70  39  26  21  78  99  56   8  50  67  45   7
 19  88  73  48  79  97  64  90  72  16 100  61  16  18  22  52  54  58  26  85
  5  74  32  59  30  55  36  77  29  79  32  39  13  24  78  36  97  70  72  23
 46  14  96  94   6  31  88   8  75  83  63  66  18  88  27  15  60  44  34  14
 79  62   5  13  50  30  80  80  66  45  99  38  93  27  31   4  72  70  37  29
 21  12  40  26  24  20  54  16  76  94  68  38  59  90  43  53  66  71  68  40
 90  21   9  65  31  43  58   4   7  47  74  45  85  26  93   8  20  20  23  70
 66  66   4  59  34  69  96  60  21  38  87   5  66  81   9  31  82  27  88   1
 59  98  94  42  97  52  67  73  66  46  28   9  38  63  70  50  27   8  58  32
 47  48  60  79  98  14  92  40  52  31  69  76  39  18  39  63  50  62  13  17
  4  80  90  97  60   1   4  10  67  95  78  39  17   8   8  86  73  25  82   5
 47  34  14  89 100  69  71  85   4  45  31  39  33   4  13  54  32  21  68  49
 73  18  84  95  69  86  21   1  76  44  95  28  49  84  90  40  49  69  91   2
 71  44  53  53  83  99  49  93  10  11  25   8  38  69  91  14  32  99   1  13
 10  96  95  87  44  13  65  20  42  72  63  15  40  83  60  48  14  45   1  72
  3  30   7   6  50  12  84  27  90  94  76  85   8  17  21  76  95  74  75  54
  1  28  39  85  73  91  38  19  23  42  35  65  25  94  83  28  52  71  23  87
 88  16 100  46  93  63   6  49  25  27  33  77  23  14  10  31   7  28  50  59
 54  46   5  39  17  55  20  78  60  37  16  20  48   8  35  64  54  89  62  61
 52  14  62  82  55  63  17   9   7  56  53  35  86  56  59  14  87  38  46  72
Numbers read: 500
```

Exercise 2 - Multiple files

Source screenshot - part 1 of 2. Read 1,500 values from three files into an ArrayList.

PDI / LAB 04 JAVA SOURCE

src/Exercise2.java | lines 1-24

```
1  import java.io.BufferedReader;
2  import java.io.BufferedWriter;
3  import java.io.IOException;
4  import java.nio.charset.StandardCharsets;
5  import java.nio.file.Files;
6  import java.nio.file.Path;
7  import java.util.ArrayList;
8  import java.util.Collections;
9  import java.util.Random;
10 import java.util.Scanner;
11
12 /** Creates n CSV files and finds extrema from the numbers read into an ArrayList. */
13 public class Exercise2 {
14     private static final int NUMBERS_PER_FILE = 500;
15
16     public static void main(String[] args) throws IOException {
17         Scanner scanner = new Scanner(System.in, StandardCharsets.UTF_8);
18         int n = ConsoleInput.readPositiveInt(scanner, "Number of files: ");
19         Path folder = Path.of("Ex2LuckyNumber");
20         Files.createDirectories(folder);
21         Random random = new Random();
22         for (int i = 1; i <= n; i++) {
23             Path file = folder.resolve("file" + i + ".csv");
24             try (BufferedWriter writer = Files.newBufferedWriter(file, StandardCharsets.UTF_8)) {
```

Exercise 2 - Multiple files

Source screenshot - part 2 of 2. Read 1,500 values from three files into an ArrayList.

PDI / LAB 04 JAVA SOURCE

src/Exercise2.java | lines 25-48

```
25                  for (int j = 0; j < NUMBERS_PER_FILE; j++) {
26                      writer.write(Integer.toString(random.nextInt(100) + 1));
27                      writer.newLine();
28                  }
29              }
30          }
31
32          ArrayList<Integer> numbers = new ArrayList<>();
33          // Read exactly the n files created by this run, ignoring older extra files.
34          for (int i = 1; i <= n; i++) {
35              Path file = folder.resolve("file" + i + ".csv");
36              try (BufferedReader reader = Files.newBufferedReader(file, StandardCharsets.UTF_8)) {
37                  String line;
38                  while ((line = reader.readLine()) != null) {
39                      numbers.add(Integer.parseInt(line));
40                  }
41              }
42          }
43          System.out.println("Files created: " + n);
44          System.out.println("Numbers loaded into ArrayList: " + numbers.size());
45          System.out.println("Minimum: " + Collections.min(numbers));
46          System.out.println("Maximum: " + Collections.max(numbers));
47      }
48  }
```

Exercise 2 - Multiple files

Program output screenshot. Read 1,500 values from three files into an ArrayList.

PDI / LAB 04

CAPTURED CONSOLE OUTPUT

Exercise2 | successful run

```
$ java -cp build Exercise2

INPUT
3

CAPTURED STDOUT
Number of files: Files created: 3
Numbers loaded into ArrayList: 1500
Minimum: 1
Maximum: 100
```

Exercise 3 - Save students

Source screenshot. Enter three fictional demonstration team members.

PDI / LAB 04 JAVA SOURCE

src/Exercise3.java | lines 1-16

```java
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Scanner;

/** Prompts for three team members and saves their name, ID, and gender as CSV. */
public class Exercise3 {
    public static void main(String[] args) throws IOException {
        Scanner scanner = new Scanner(System.in, StandardCharsets.UTF_8);
        ArrayList<Student> students = StudentData.input(scanner, 3);
        Path file = Path.of("Ex3StudentData.csv");
        StudentData.write(file, students);
        System.out.println("Saved " + students.size() + " students to " + file);
    }
}
```

Exercise 3 - Save students

Program output screenshot. Enter three fictional demonstration team members.

PDI / LAB 04

CAPTURED CONSOLE OUTPUT

Exercise3 | successful run

```
$ java -cp build Exercise3

INPUT (fictional demonstration students)
Zoe, Chan
001
female
Bora "Bo"
002
male
Ana
003
female

CAPTURED STDOUT
Student 1 of 3
Name: ID: Gender: Student 2 of 3
Name: ID: Gender: Student 3 of 3
Name: ID: Gender: Saved 3 students to Ex3StudentData.csv
```

Exercise 4 - Read students

Source screenshot. Reconstruct students in an ArrayList and display them.

PDI / LAB 04 JAVA SOURCE

src/Exercise4.java | lines 1-15

```java
import java.io.IOException;
import java.nio.file.Path;
import java.util.ArrayList;

/** Loads student objects from CSV into an ArrayList, then displays that list. */
public class Exercise4 {
    public static void main(String[] args) throws IOException {
        Path file = args.length > 0 ? Path.of(args[0]) : StudentData.defaultInput();
        ArrayList<Student> students = StudentData.read(file);
        System.out.println("Loaded " + students.size() + " students from " + file);
        for (Student student : students) {
            System.out.println(student);
        }
    }
}
```

Exercise 4 - Read students

Program output screenshot. Reconstruct students in an ArrayList and display them.

PDI / LAB 04

CAPTURED CONSOLE OUTPUT

Exercise4 | successful run

```
$ java -cp build Exercise4

CAPTURED STDOUT
Loaded 3 students from Ex3StudentData.csv
Name: Zoe, Chan | ID: 001 | Gender: female
Name: Bora "Bo" | ID: 002 | Gender: male
Name: Ana | ID: 003 | Gender: female
```

Exercise 5 - Add and sort

Source screenshot. Add three demonstration students and save six names in A-Z order.

PDI / LAB 04 JAVA SOURCE

src/Exercise5.java | lines 1-25

```java
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.Scanner;

/** Adds three students, sorts the ArrayList by name, and saves StudentData.csv. */
public class Exercise5 {
    public static void main(String[] args) throws IOException {
        Path input = args.length > 0 ? Path.of(args[0]) : StudentData.defaultInput();
        Path output = args.length > 1 ? Path.of(args[1]) : Path.of("StudentData.csv");
        ArrayList<Student> students = StudentData.read(input);
        System.out.println("Loaded " + students.size() + " students from " + input);
        Scanner scanner = new Scanner(System.in, StandardCharsets.UTF_8);
        students.addAll(StudentData.input(scanner, 3));
        students.sort(Comparator.comparing(Student::name, String.CASE_INSENSITIVE_ORDER)
                .thenComparing(Student::name).thenComparing(Student::id));
        StudentData.write(output, students);
        System.out.println("Saved " + students.size() + " students sorted by name (A-Z) to " + output);
        for (Student student : students) {
            System.out.println(student);
        }
    }
}
```

Exercise 5 - Add and sort

Program output screenshot. Add three demonstration students and save six names in A-Z order.

PDI / LAB 04

CAPTURED CONSOLE OUTPUT

Exercise5 | successful run

```
$ java -cp build Exercise5

INPUT (fictional demonstration students)
dara
006
male
Chenda
005
female
Bopha
004
female

CAPTURED STDOUT
Loaded 3 students from Ex3StudentData.csv
Student 1 of 3
Name: ID: Gender: Student 2 of 3
Name: ID: Gender: Student 3 of 3
Name: ID: Gender: Saved 6 students sorted by name (A-Z) to StudentData.csv
Name: Ana | ID: 003 | Gender: female
Name: Bopha | ID: 004 | Gender: female
Name: Bora "Bo" | ID: 002 | Gender: male
Name: Chenda | ID: 005 | Gender: female
Name: dara | ID: 006 | Gender: male
Name: Zoe, Chan | ID: 001 | Gender: female
```

Exercise 6 - Candidate analysis

Source screenshot. Analyze the supplied local dataset; publish aggregate output only.

PDI / LAB 04 JAVA SOURCE

src/Exercise6.java | lines 1-34

```java
import java.io.IOException;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Map;

/** Displays the candidate CSV and answers all four analysis questions. */
public class Exercise6 {
    public static void main(String[] args) throws IOException {
        Path file = args.length > 0 && !args[0].equals("--summary")
                ? Path.of(args[0]) : CandidateData.DEFAULT_FILE;
        CandidateData data = CandidateData.read(file);
        if (!Arrays.asList(args).contains("--summary")) {
            data.display();
        }
        ArrayList<Candidate> candidates = data.candidates();
        CandidateStatistics.printGenderSummary("All candidates", candidates);
        CandidateStatistics.printGenderSummary("Average score >= 90",
                CandidateStatistics.averageAtLeast(candidates, 90));
        System.out.println("Math score >= 85: "
                + CandidateStatistics.subjectAtLeast(candidates, Candidate::math, 85));
        System.out.println("English score >= 85: "
                + CandidateStatistics.subjectAtLeast(candidates, Candidate::english, 85));
        System.out.println("Computer score >= 85: "
                + CandidateStatistics.subjectAtLeast(candidates, Candidate::computer, 85));
        System.out.println("All three subjects >= 85: "
                + CandidateStatistics.allSubjectsAtLeast(candidates, 85));
        System.out.println();
        for (Map.Entry<String, ArrayList<Candidate>> group
                : CandidateStatistics.byGroup(candidates).entrySet()) {
            CandidateStatistics.printGenderSummary("Group " + group.getKey(), group.getValue());
        }
    }
}
```

Exercise 6 - Candidate analysis

Program output screenshot - part 1 of 2. Analyze the supplied local dataset; publish aggregate output only.

PDI / LAB 04

CAPTURED CONSOLE OUTPUT

Exercise6 | successful run | part 1/2

```
$ java -cp build Exercise6 data/CandidatePerformanceTest.csv --summary

CAPTURED STDOUT
All candidates
Total candidates: 958
Male candidates: 467 (48.75%)
Female candidates: 491 (51.25%)

Average score >= 90
Total candidates: 48
Male candidates: 14 (29.17%)
Female candidates: 34 (70.83%)

Math score >= 85: 112
English score >= 85: 142
Computer score >= 85: 128
All three subjects >= 85: 58

Group A
Total candidates: 81
Male candidates: 50 (61.73%)
Female candidates: 31 (38.27%)
```

Exercise 6 - Candidate analysis

Program output screenshot - part 2 of 2. Analyze the supplied local dataset; publish aggregate output only.

PDI / LAB 04

CAPTURED CONSOLE OUTPUT

Exercise6 | successful run | part 2/2

```
Group B
Total candidates: 185
Male candidates: 84 (45.41%)
Female candidates: 101 (54.59%)

Group C
Total candidates: 308
Male candidates: 134 (43.51%)
Female candidates: 174 (56.49%)

Group D
Total candidates: 249
Male candidates: 130 (52.21%)
Female candidates: 119 (47.79%)

Group E
Total candidates: 135
Male candidates: 69 (51.11%)
Female candidates: 66 (48.89%)
```

Exercise 7 - Group exports

Source screenshot - part 1 of 2. Export every group and append total and gender summaries.

PDI / LAB 04 JAVA SOURCE

src/Exercise7.java | lines 1-31

```java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.Map;

/** Exports every group with a header, original records, and three summary rows. */
public class Exercise7 {
    public static void main(String[] args) throws IOException {
        Path input = args.length > 0 && !args[0].equals("--summary")
                ? Path.of(args[0]) : CandidateData.DEFAULT_FILE;
        Path output = args.length > 1 && !args[1].equals("--summary")
                ? Path.of(args[1]) : Path.of("groups");
        CandidateData data = CandidateData.read(input);
        if (!Arrays.asList(args).contains("--summary")) {
            data.display();
        }
        Map<String, ArrayList<Candidate>> groups = CandidateStatistics.byGroup(data.candidates());
        // Always create the four files requested by the handout, even for empty groups.
        for (String group : List.of("A", "B", "C", "D")) {
            groups.computeIfAbsent(group, key -> new ArrayList<>());
        }
        Files.createDirectories(output);
        int exported = 0;
        for (Map.Entry<String, ArrayList<Candidate>> entry : groups.entrySet()) {
            List<Candidate> members = entry.getValue();
            ArrayList<List<String>> rows = new ArrayList<>();
```

Exercise 7 - Group exports

Source screenshot - part 2 of 2. Export every group and append total and gender summaries.

PDI / LAB 04 JAVA SOURCE

src/Exercise7.java | lines 32-61

```
32              rows.add(data.header());
33              for (Candidate candidate : members) {
34                  rows.add(candidate.row());
35              }
36              int width = data.header().size();
37              rows.add(summaryRow(width, "Summary - total students", Integer.toString(members.size())));
38              rows.add(summaryRow(width, "Summary - male (%)", percent(members, "male")));
39              rows.add(summaryRow(width, "Summary - female (%)", percent(members, "female")));
40              Path file = output.resolve("group" + entry.getKey() + ".csv");
41              Csv.write(file, rows);
42              CandidateStatistics.printGenderSummary("Group " + entry.getKey(), members);
43              System.out.println("Wrote " + file);
44              System.out.println();
45              exported += members.size();
46          }
47          System.out.println("Exported " + exported + " candidates to " + groups.size() + " group files.");
48      }
49
50      private static List<String> summaryRow(int width, String label, String value) {
51          ArrayList<String> row = new ArrayList<>(Collections.nCopies(width, ""));
52          row.set(0, label);
53          row.set(1, value);
54          return row;
55      }
56
57      private static String percent(List<Candidate> candidates, String gender) {
58          return String.format(Locale.ROOT, "%.2f%%", CandidateStatistics.percentage(
59                  CandidateStatistics.countGender(candidates, gender), candidates.size()));
60      }
61  }
```

Exercise 7 - Group exports

Program output screenshot - part 1 of 2. Export every group and append total and gender summaries.

PDI / LAB 04

CAPTURED CONSOLE OUTPUT

Exercise7 | successful run | part 1/2

```
$ java -cp build Exercise7 data/CandidatePerformanceTest.csv groups --summary

CAPTURED STDOUT
Group A
Total candidates: 81
Male candidates: 50 (61.73%)
Female candidates: 31 (38.27%)

Wrote groups\groupA.csv

Group B
Total candidates: 185
Male candidates: 84 (45.41%)
Female candidates: 101 (54.59%)

Wrote groups\groupB.csv

Group C
Total candidates: 308
Male candidates: 134 (43.51%)
Female candidates: 174 (56.49%)
```

Exercise 7 - Group exports

Program output screenshot - part 2 of 2. Export every group and append total and gender summaries.

PDI / LAB 04

CAPTURED CONSOLE OUTPUT

Exercise7 | successful run | part 2/2

```
Wrote groups\groupC.csv

Group D
Total candidates: 249
Male candidates: 130 (52.21%)
Female candidates: 119 (47.79%)

Wrote groups\groupD.csv

Group E
Total candidates: 135
Male candidates: 69 (51.11%)
Female candidates: 66 (48.89%)

Wrote groups\groupE.csv

Exported 958 candidates to 5 group files.
```

Bonus - CSV object round trip

Source screenshot. Serialize Student objects and verify the reconstructed objects.

PDI / LAB 04 JAVA SOURCE

src/BonusExercise.java | lines 1-34

```java
import java.io.IOException;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

/** Serializes Student objects as CSV, then reconstructs and checks those objects. */
public class BonusExercise {
    public static void serialize(List<Student> students, Path file) throws IOException {
        // Converts object fields to CSV strings, with escaping handled by Csv.write.
        StudentData.write(file, students);
    }

    public static ArrayList<Student> deserialize(Path file) throws IOException {
        // Parses CSV strings and constructs new Student instances, preserving text IDs.
        return StudentData.read(file);
    }

    public static void main(String[] args) throws IOException {
        Path input = args.length > 0 ? Path.of(args[0]) : StudentData.defaultInput();
        Path output = args.length > 1 ? Path.of(args[1]) : Path.of("BonusStudents.csv");
        ArrayList<Student> original = StudentData.read(input);
        serialize(original, output);
        System.out.println("Serialized " + original.size() + " Student objects to " + output);
        ArrayList<Student> restored = deserialize(output);
        System.out.println("Deserialized " + restored.size() + " Student objects:");
        for (Student student : restored) {
            System.out.println(student);
        }
        if (!original.equals(restored)) {
            throw new IllegalStateException("CSV round-trip verification failed.");
        }
        System.out.println("Round-trip verified: every restored object equals its original.");
    }
}
```

Bonus - CSV object round trip

Program output screenshot. Serialize Student objects and verify the reconstructed objects.

PDI / LAB 04

CAPTURED CONSOLE OUTPUT

BonusExercise | successful run

```
$ java -cp build BonusExercise

CAPTURED STDOUT
Serialized 6 Student objects to BonusStudents.csv
Deserialized 6 Student objects:
Name: Ana | ID: 003 | Gender: female
Name: Bopha | ID: 004 | Gender: female
Name: Bora "Bo" | ID: 002 | Gender: male
Name: Chenda | ID: 005 | Gender: female
Name: dara | ID: 006 | Gender: male
Name: Zoe, Chan | ID: 001 | Gender: female
Round-trip verified: every restored object equals its original.
```

Verification - integration checks

Program output screenshot. Independent checks compare Java output with Python CSV calculations.

PDI / LAB 04

CAPTURED CONSOLE OUTPUT

python tests/verify.py | all checks passed

```
PASS Exercise 1: 500 saved numbers match the displayed file contents
PASS Exercise 2: file counts, input validation, stale files, and exact extrema
PASS Exercise 3: three students, required input, CSV escaping, and leading-zero IDs
PASS Exercise 4: ArrayList display, file-name fallback, BOM, and multiline fields
PASS Exercise 5: append exactly three, sort, save, and reread six students
PASS original local dataset: 958 candidates (not published)
PASS Exercise 6: threshold boundaries, groups, display, and empty populations
PASS Exercise 7: all groups, lossless partition, rectangular CSV, footer statistics, empty groups
PASS Bonus: object round trips preserve escaped fields, multiline names, and text IDs
```